Organiser le delivery logiciel AI-native avec Gas Town

Agile En Seine & IA

Jean-Louis Rigau & Emmanuel Sciara

Mardi 22 septembre 2026

#Engineering AI-native #Delivery distribué #Orchestration d'agents #Exécution supervisée

Ouvrir la présentation

Intervenants



INTERVENANT

Jean-Louis Rigau

Vibe coder & AI-native advisor. J'aide les organisations et les équipes engineering à faire évoluer leurs plateformes et leurs workflows de delivery pour intégrer durablement les systèmes multi-agents et les pratiques agentiques.

linkedin.com/in/jlrigau



INTERVENANT

Emmanuel Sciara

Agent Lead & ingénieur AI-native. J'interviens sur la stratégie d'entreprise, le déploiement et le coaching de l'utilisation des coding agents, avec une spécialisation Claude Code.

linkedin.com/in/emmanuelsciara

Nos partenaires

Merci à celles et ceux qui rendent l'événement possible.



AgilenSeine

Les 8 étapes du développement IA

1



IA nulle ou quasi nulle

2



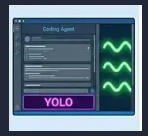
Assistant IDE avec permissions

3



Agent IDE en mode YOLO

4



Agent IDE en mode large

5



Agent CLI en session unique

6



Agent CLI en multi-sessions

7



10+ agents, pilotés à la main

8



Construire un orchestrateur

CHANGEMENT DE PRATIQUE

D'une requête à un assistant vers la gestion d'un système de delivery.

- Les premières étapes améliorent le flux individuel.
- Les étapes intermédiaires augmentent le débit (CLI).
- Les dernières étapes créent un enjeu de coordination.
- Gas Town entre en jeu quand le pilotage manuel n'est plus gérable.

TAKEAWAY
Le développement avec l'IA évolue de l'assistance individuelle vers la coordination organisationnelle.

Steve Yegge



Photo : Bram Cymet, CC BY-NC 2.0

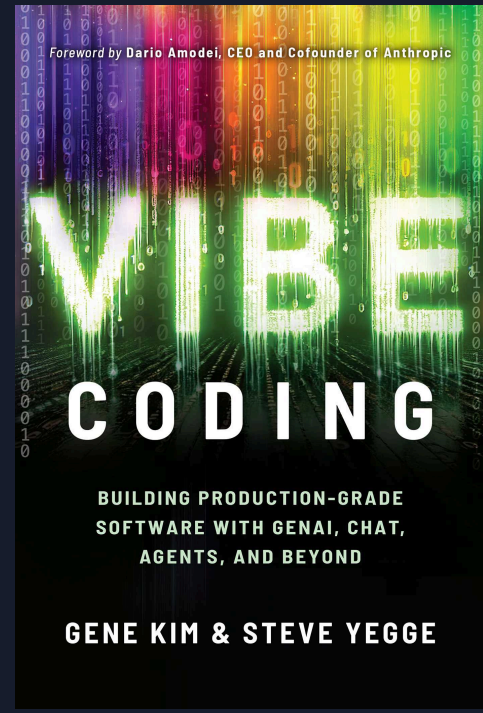
DE LA PRATIQUE À L'ORGANISATION

Steve Yegge cadre la bascule de l'utilisation individuelle des agents de code au delivery logiciel AI-native.

Vibe Coding
Developpement de logiciel de qualité production avec des agents de coding IA.

Gas Town
Un modèle opérationnel pour coordonner de nombreux agents sur des projets réels.

Beads
Un registre de travail basé sur Git pour workflows d'ingénierie agentiques.



Couverture : IT Revolution / Simon & Schuster

Gas Town : un système de delivery urbanisé

TOWN

Quartier général agentique

Un espace opérationnel séparé où vivent orchestration, identité, mail et traçage du travail.

MAYOR

Coordination

C'est le point d'entrée : absorbe le bruit des workers et transforme l'intention en travail coordonné.

RIGS (TOUR DE FORAGE)

Usines projets

Polecats, Crew, Witness et Refinery opèrent directement sur des repositories.

LA VILLE COMME MODÈLE OPÉRATIONNEL

Gas Town rend le vibe coding opérationnel sans nier le chaos.

- Les agents deviennent des workers visibles, avec une identité.
- Les beads fournissent un registre du travail, de la mémoire, du mail et de la provenance, avec Git comme backend.
- Les convoys suivent le delivery ; les Polecats exécutent.
- Witness et Refinery évitent le chaos du merge.

TAKEAWAY

Gas Town rend l'utilisation des agents de code observable et coordonnée.

Pourquoi les workflows single-agent cassent

AGENT UNIQUE

Très bon assistant local

Un seul contexte, un seul fil d'exécution, une seule branche de travail.

01

Saturation du contexte

Les contraintes importantes disparaissent la session grandit.

02

Exécution séquentielle

Le débit est limité par une seule boucle de worker.

03

Pas de mécanisme de coordination

Les dépendances et les responsabilités restent implicites.

04

Pression sur le merge

Les changements parallèles deviennent difficiles à intégrer proprement.

05

Pas de supervision opérationnelle

Le travail bloqué devient difficile à observer et à récupérer.

PROBLÈME

le développement avec IA se fait localement, alors que le delivery logiciel est organisationnel.

- Les Context Windows ne modélisent pas les dépendances.
- Le travail séquentiel limite le débit.
- Le travail parallèle crée une pression sur l'intégration.
- La supervision devient un sujet de premier ordre.

TAKEAWAY

Le delivery logiciel est organisationnel. Le goulot devient la coordination.

Gas Town organise le travail, pas les prompts

ENTRÉE

Intention humaine

Une demande en langage naturel, incomplète par construction.

COUCHE D'ORGANISATION

Gas Town structure le delivery

- **Mayor** planifie et coordonne.
- **Beads** rendent le travail adressable.
- **Convoys** suivent les flux de delivery.
- **Witness & Refinery** gardent l'exécution observable.
- **Polecats** font le travail.

SORTIE

Delivery observable

Statuts, dépendances, responsabilités, progression et flux de merge deviennent inspectables.

RÉPONSE

Gas Town transforme l'intention en objets de delivery partagés.

- Le Mayor sépare planification et exécution.
- Beads et convoys rendent le travail adressable et traçable.
- Witness et Refinery rendent la supervision et la convergence visibles.
- Les Polecats exécutent.

TAKEAWAY

Le passage à l'échelle vient de la coordination partagée, pas de prompts plus gros.

L'organisation

TOWN

- **Overseer** — Intention et contrôle humains
- **Mayor** — Coordination cross-rig
- **Deacon** — Santé système

RIG

Un repository projet avec son propre environnement d'exécution.

- **Crew** — Workers persistants
- **Polecats** — Workers éphémères
- **Witness** — Supervision du rig
- **Refinery** — Queue de merge

POURQUOI C'EST IMPORTANT

Gas Town s'articule autour d'une town et de ses rigs.

- La Town coordonne les repositories.
- Chaque Rig cible une vraie codebase.
- Les Crew sont persistants; les Polecats sont des workers d'exécution jetables.
- Witness et Refinery opèrent au niveau du rig.

TAKEAWAY

Gas Town sépare coordination, exécution, supervision et intégration.

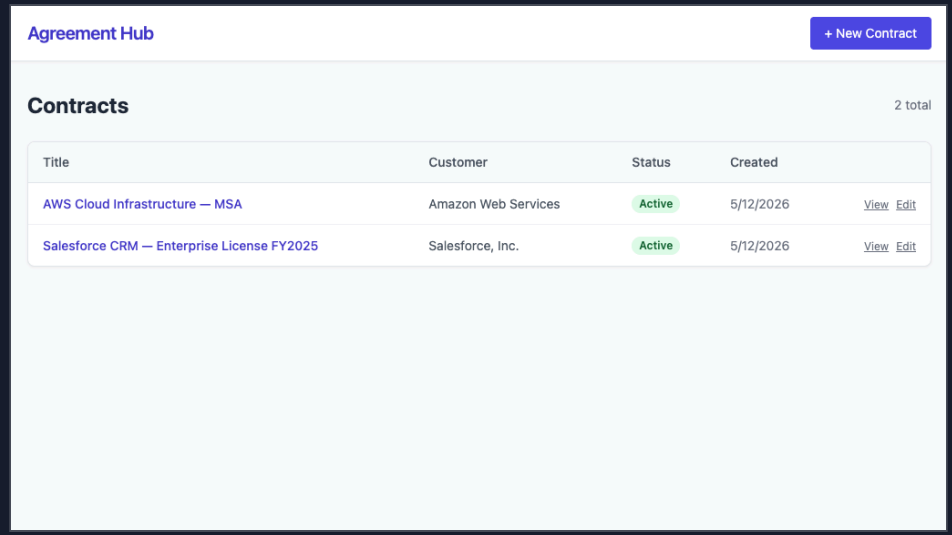
Application de l'atelier

AGREEMENT HUB

Agreement Hub est une application de Contract Lifecycle Management. Elle permet de créer, consulter et gérer des contrats.

- Le Mayor choisira deux fonctionnalités à ajouter à l'application existante.
- Vous explorerez une copie locale de l'application.
- Nous la mettrons à jour après le travail des agents pour découvrir le résultat.

> Côté technique : React/Vite, Express/TypeScript et SQLite.

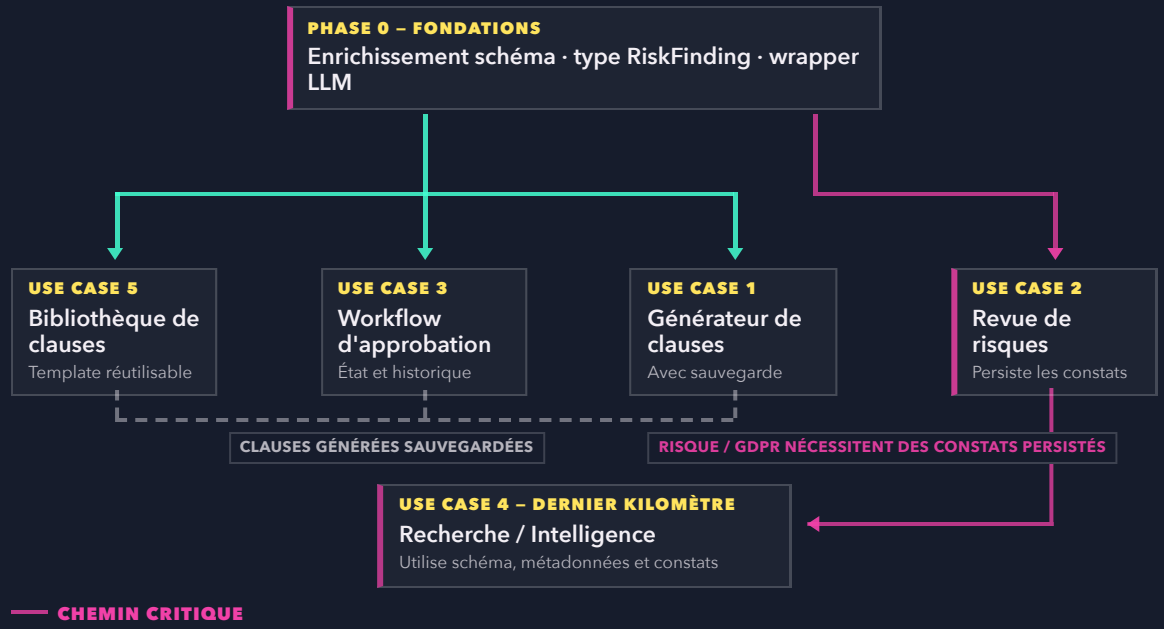


LES USE CASES À EXPLORER

Retrouvez les cinq use cases depuis le `README` du dépôt.

GitHub ↗

Beads : découpage du travail & dépendances



POURQUOI C'EST IMPORTANT

Les beads transforment un besoin en travail exécutable, avec ordre, responsabilités et statut.

- Les dépendances exposent le chemin critique.
- Le travail qui est prêt peut être exécuté immédiatement.
- Le travail bloqué devient visible au lieu de rester implicite.
- Les hooks gardent les workers connectés aux affectations actives.

TAKEAWAY
Les beads montrent ce qui peut démarrer maintenant, ce qui est bloqué, et pourquoi.

Convoys : flux de delivery



POURQUOI C'EST IMPORTANT

Les convoys rendent chaque use case visible comme flux de delivery.

- Un même numéro d'étape signifie du travail parallèle.
- La progression devient visible entre workers.
- Référence partagée entre les rôles.

TAKEAWAY

Les convoys transforment le graphe de beads en flux de delivery observables.

Polecats : exécution parallèle

POLECATS	BEADS
🐱 FURIOSA	Fondations Bibliothèque de clauses Revue de risques
🐱 NUX	Workflow d'approbation contrat
🐱 SLIT	Générateur de clauses Recherche / Intelligence MVP

UNE DEMANDE PRODUIT
↓
BEADS CADRÉS
↓
WORKERS D'EXÉCUTION ISOLÉS (POLECATS)
↓
DELIVERY COORDONNÉ

POURQUOI C'EST IMPORTANT

Le Mayor assigne des beads cadrés à des Polecats isolés, chacun dans son contexte et sa branche.

- Lancés à la demande.
- Les affectations de travail restent isolées entre elles.
- Les Polecats disparaissent après livraison.
- La parallélisation augmente le débit, pas le chaos.

TAKEAWAY
Les Polecats scalent l'exécution via des contextes isolés.

La supervision garde l'exécution opérationnelle



POURQUOI C'EST IMPORTANT

La supervision Gas Town est organisée en couches, pas autour d'un watcher unique.

- Le Daemon réveille le Boot pour vérifier la santé.
- Le Deacon garde la town en bonne santé.
- Le Witness maintient l'exécution du rig.
- la Refinery maintient la convergence.

TAKEAWAY
La supervision rend l'exécution distribuée opérationnelle.

À vous de piloter Gas Town

Deux évolutions, un plan, des tâches coordonnées, une application vérifiée.

Un point de départ commun

L'application actuelle, deux incréments choisis par le Mayor, puis validés par votre binôme.

Garder la main

Observer les agents, comprendre leur avancement et intervenir si nécessaire.

Vérifier le résultat

Lancer l'application et tester ce qui a réellement été livré.

Travaillez par deux sur un lab commun.
L'un manipule, l'autre suit les étapes et vérifie. Inversez les rôles pendant l'atelier.



Installer l'environnement

Taxiway permet de créer un environnement local isolé pour l'atelier.

taxiway.sh ↗

Prérequis		
WSL2	Sous Windows 11, installez WSL2, puis utilisez son terminal Linux.	WSL2 ↗
Docker	Installez Docker Desktop sur macOS ou Docker Engine avec le plugin Compose sous Linux ou WSL2.	Docker ↗
Lima	Installez Lima, que Taxiway utilisera comme driver.	Lima ↗

Exécutez ces commandes sur votre machine (dans WSL2 sous Windows).

Installer Taxiway

```
curl -fsSL https://taxiway.run | sh
```

Initialiser et vérifier l'environnement

```
taxiway init
taxiway status
```

Préparer les outils de l'application

Prérequis		
Git	Récupérer le dépôt et ses changements.	Git ↗
Node.js + npm	Node.js 24 LTS et npm 11+ pour lancer l'application.	Node.js ↗
Claude Code	Générer les données et vérifier l'application.	Claude Code ↗

Exécutez ces commandes sur votre machine (dans WSL2 sous Windows).

Installer agent-browser

```
npm install -g agent-browser
```

Installer le navigateur — macOS

```
agent-browser install
```

Installer le navigateur — Linux / WSL2

```
agent-browser install --with-deps
```

Préparer le skill agent-browser

```
npx skills add vercel-labs/agent-browser \
  --global --skill agent-browser \
  --agent claude-code
```

Se connecter à Claude Code

```
claude
```

> Validez la connexion à Claude Code, puis quittez avec `/exit`.

Créer le lab Taxiway

Depuis la machine hôte (WSL2 sous Windows), créez un lab sur l'application actuelle.

Créer le lab

```
taxiway up agile-en-seine --type gastown \
  --repo https://github.com/manufacture-dev/agreement-hub.git \
  --set version=1.1.0 \
  --set beads-version=1.0.3 \
  --set model=claude-opus-4-8
```

Vérifier que le lab est prêt

```
taxiway status
taxiway doctor agile-en-seine
```

> En cas d'erreur, relancez la commande avec `--fix`. Quelques warnings non bloquants peuvent rester.

Récupérer les accès aux services

```
taxiway access
```

Connectez-vous à Langfuse et ouvrez le projet du lab.

Ouvrir Langfuse ↗

CHECKPOINT SETUP

Vous pouvez continuer si le lab est accessible, si les diagnostics ne signalent aucune erreur bloquante et si vous avez pu vous connecter à Langfuse.

En cas de blocage, demandez de l'aide.

Récupérer et lancer l'application

Sur votre machine, récupérez l'application depuis le lab Taxiway.

Récupérer l'application depuis le lab

```
git clone ~/.taxiway/lab-state/agile-en-seine/git/agreement-hub.git ~/agreement-hub
```

> Cette copie locale permettra de récupérer les évolutions produites dans le lab.

Ouvrez deux terminaux : un pour le backend et un pour le frontend.

Lancer le backend

```
cd ~/agreement-hub/backend
npm install
npm run dev
```

Lancer le frontend

```
cd ~/agreement-hub/frontend
npm install
npm run dev
```

DÉCOUVRIR L'APPLICATION

Gardez les terminaux ouverts, puis explorez l'application avant toute réalisation.

Ouvrir l'app ↗

Générer les premiers contrats

Générez des données dans l'application pour permettre de tester les fonctionnalités qui seront livrées.

Ouvrir Claude Code dans l'application

```
cd ~/agreement-hub
claude
```

> Gardez cette session ouverte : vous reprendrez le même agent pour vérifier les fonctionnalités livrées.

Agent application **Créer le jeu de données** Copier

```
/agent-browser Ouvre http://localhost:5173 en mode visible
(headed), dans une session dédiée. Garde la même fenêtre
et le même onglet.

Inspecte l'application, puis crée 12 contrats fictifs
réalistes via l'interface. Varie les types, entreprises,
statuts et clauses pour couvrir différents cas.
Utilise des titres métier naturels, sans préfixe
ni nom de client. Garde les données cohérentes.

Respecte les champs et validations de l'application.
Ne modifie ni le code ni les contrats existants ;
pas de SQL ni d'appel direct à l'API.

Vérifie la persistance après rechargement. En cas
d'interruption, reprends sans doublons jusqu'à 12 créations.
Résume les contrats créés et les éventuels blocages.
```

Entrer dans le lab et naviguer avec tmux

Enregistrez la session pour la rejouer et l'analyser, puis entrez dans le lab et rejoignez le Mayor.

Démarrer l'enregistrement

```
taxiway record start agile-en-seine
```

Entrer dans le lab

```
taxiway shell agile-en-seine
```

Vérifier Gas Town

```
gt doctor
gt status
```

Rejoindre le Mayor

```
gt mayor attach
```

Une fois attaché au Mayor, utilisez les raccourcis imbriqués à droite.

Les sessions tmux de Gas Town s'ouvrent à l'intérieur de la session tmux de Taxiway.

TMUX COURANT

`Ctrl` + `b` → `s`	Lister les sessions Taxiway
`Ctrl` + `b` → `d`	Sortir de Taxiway

`Ctrl+b`, relâchez les touches, puis tapez la commande.

TMUX IMBRIQUÉ

`Ctrl` + `b` → `Ctrl` + `b` → `s`	Lister les sessions Gas Town
`Ctrl` + `b` → `Ctrl` + `b` → `d`	Revenir à Taxiway

`Ctrl+b` deux fois, relâchez les touches, puis tapez la commande.

Guide officiel tmux ↗ **Cheat sheet tmux ↗**

Confier la mission au Mayor

Le Mayor inspecte l'application et choisit deux incréments réalisables en parallèle.

RÉSULTAT ATTENDU

Un plan court : résultats visibles, dépendances, fichiers partagés, vérifications et hors périmètre.

Le Mayor attend votre GO.

Les fonctionnalités retenues pourront être différentes d'un binôme à l'autre.

Mayor **Définir la mission** Copier

```
Dans agreement_hub, lis docs/use-cases/README.md et inspecte l'app.
Choisis exactement 2 des 5 use cases listés, aucun hors liste.
Pour chacun : tranche minimale fidèle au besoin, testable dans
l'UI, indépendante ; ajuste choix et scope pour <15 min après GO.
Plan : numéros et noms exacts, couvert/exclu, résultats,
dépendances, fichiers partagés et checks.
Use cases IA (#1/#2) éligibles : simule le LLM sans service externe.
Attends mon GO. Décide seul des choix courants ; demande
pour une extension, un service externe ou un risque de perte.

Après GO, coordonne sans coder : beads produit, convoy,
Polecats distincts et worktrees isolés. Un seul responsable
par fichier partagé. Intègre via Witness et Refinery,
sans push direct sur main ni artefacts runtime committés.

À 8 min, réduis le scope si nécessaire ; à 12 min, stabilise.
Toutes les 3 min ou si blocage : résultat, état Gas Town,
blocages et arbitrages. Poursuis le plan sans nouvelle validation.
Vérifie puis clos les beads intégrés. Bilan : livré, démontrable,
preuves, branche et SHA, checks, différé et risques.
Ensuite, arrête le monitoring.
```

Lancer et observer les agents au travail

Envoyez le GO au Mayor, puis utilisez `Ctrl` + `b` → `Ctrl` + `b` → `s` pour choisir une session.

Mayor **Autoriser l'exécution** Copier

```
GO
```

MAYOR
Le Mayor présente l'avancement toutes les trois minutes ou en cas de blocage. Revenez le voir pour lire le bilan final.

POLECAT
Suivez l'implémentation et les tests d'une fonctionnalité.

WITNESS
Observez la surveillance des Polecats et la détection des blocages.

REFINERY
Suivez l'intégration des changements livrés par les Polecats.

DEACON
Observez les patrouilles de santé de Gas Town et la surveillance des Witnesses.

Observez sans saisir d'instructions dans les agents.

Suivre la réalisation

Détachez-vous de la session Gas Town avec `Ctrl` + `b` → `Ctrl` + `b` → `d`.

Observer l'état de Gas Town

```
gt status
```

Repérez l'arrivée des Polecats et l'état des agents.

Consulter les messages reçus

```
gt mail inbox mayor/
```

À tout moment, revenez aux agents avec `gt mayor attach`, puis choisissez une session. Détachez-vous à nouveau pour retrouver les commandes de suivi.

Explorer les tâches

```
bd list --all --flat
bd ready
bd show <bead_id>
```

Suivre l'avancement

```
gt convoy list --all
gt convoy status <convoy_id>
```

Remplacez `<bead_id>` et `<convoy_id>` par les identifiants affichés.

Suivre l'intégration

```
gt refinery queue agreement_hub
```

La file de la Refinery indique les changements en attente d'intégration.

CHECKPOINT RÉALISATION

Alternez les rôles et suivez les tâches, les Polecats et les convoys : aucun nouveau prompt nécessaire.

C’est l’heure du bilan

Le Mayor résume les fonctionnalités réalisées, les vérifications effectuées et ce qui reste à faire.

Retrouver le bilan dans la session du Mayor

```
gt mayor attach
```

> Conservez ce bilan pour le comparer au résultat observé dans l’application.

CHECKPOINT LIVRAISON

Identifiez les fonctionnalités livrées et les éventuels points à vérifier dans l’application.

Mettre en pause et arrêter l'enregistrement

Après avoir récupéré le bilan du Mayor, mettez les agents en pause et arrêtez l'enregistrement.

Détachez-vous du Mayor avec `Ctrl` + `b` → `Ctrl` + `b` → `d` pour retrouver le shell du lab.

Geler le travail des agents

```
gt estop
```

Le lab reste disponible pour récupérer le code et consulter les traces.

Une fois les agents en pause, revenez sur la machine hôte avec `Ctrl` + `b` → `d`.

Arrêter l'enregistrement

```
taxiway record stop agile-en-seine
```

CHECKPOINT ENREGISTREMENT

Enregistrement arrêté et bilan du Mayor conservé.

Mettre à jour l'application

Sur votre machine, récupérez les changements livrés dans la copie locale de l'application.

Récupérer les changements

```
cd ~/agreement-hub
git fetch
git rebase
```

Ouvrez l'application pour voir le résultat.

Ouvrir l'app ↗

Si une erreur apparaît dans le terminal backend ou frontend, arrêtez le serveur concerné avec `Ctrl` + `c`, puis exécutez les commandes ci-dessous.

Réinstaller les dépendances et relancer le serveur

```
npm install
npm run dev
```

> Si tout fonctionne, laissez les serveurs tourner. Si l'erreur persiste, demandez de l'aide.

APPLICATION À JOUR

Les changements sont récupérés et l'application est accessible dans le navigateur.

Vérifier les fonctionnalités avec l'agent

Reprenez la session Claude Code utilisée pour créer les contrats et envoyez ce prompt.

RÉSULTAT ATTENDU

Pour chaque fonctionnalité ajoutée : ce qui a été testé, le résultat observé et les éventuels problèmes.

Suivez les tests de l'agent dans le navigateur.

Agent application **Vérifier l'application** Copier

```
/agent-browser Utilise l'historique Git et le code du dépôt
pour identifier les fonctionnalités ajoutées pendant l'atelier.

Ouvre http://localhost:5173 en mode visible (headed).
Réutilise la fenêtre et l'onglet de création des contrats.

Teste chaque fonctionnalité ajoutée via l'interface :
parcours principal, validations et persistance après
rechargement lorsque c'est pertinent.

Préserve les contrats existants. Crée des données fictives
si nécessaire, sans modifier le code ni utiliser SQL
ou des appels directs à l'API pour tester l'application.

Résume les fonctionnalités identifiées, les tests effectués,
les résultats observés et les éventuels blocages.
Signale clairement ce qui n'a pas pu être vérifié.
```

Analyser le déroulement de l’atelier

Sur la machine hôte, lancez l’analyse interactive de l’enregistrement avec Claude Code.

Analyser le dernier enregistrement

```
taxiway record analyse agile-en-seine \
  --interactive --runner claude-code \
  --language fr
```

> La session est interactive : posez vos questions à l’agent pour approfondir l’analyse et comprendre ce qui s’est passé.

Dans Langfuse, ouvrez le projet du lab pour consulter les traces et le coût des appels.

Ouvrir Langfuse ↗

Bravo !

Vous avez piloté votre premier lab Gas Town.

Exprimer le besoin

Présenter le résultat souhaité au Mayor et valider son plan.

Observer les agents

Suivre les agents et comprendre leur avancement.

Vérifier le résultat

Examiner les fonctionnalités livrées et analyser le déroulement.



Aller plus loin avec les formulas

FORMULA

Une formula transforme un pattern d'orchestration en playbook répétable : étapes, inputs, gates, parallélisme et artifacts.

EXEMPLE

mol-idea-to-plan

D'une idée vague à un plan revu et un travail prêt à devenir des beads.

Ouvrir la formula sur GitHub

TAKEAWAY

Les formulas rendent l'orchestration réutilisable : le workflow devient une partie du système de delivery.

01 · VOUS → CREW	Décrire l'idée en langage naturel; aucune structure nécessaire.
02 · INTAKE	L'agent structure l'idée en draft PRD.
03 · PRD-REVIEW	6 polecats relisent le PRD en parallèle : exigences, gaps, ambiguïtés, faisabilité, scope et stakeholders.
04 · HUMAN-CLARIFY	L'agent consolide les questions; l'humain répond avant la suite.
05 · GENERATE-PLAN	6 polecats conçoivent l'implémentation : API, data, UX, scale, sécurité et intégration.
06 · PRD-ALIGN 1-3	3 rounds × 2 Polecats alignent le plan avec le PRD et corrigent à chaque round.
07 · PLAN-REVIEW 1-3	3 rounds × 2 Polecats relisent le plan : complétude, séquencement, risque, scope creep et testabilité.
08 · CREATE-BEADS	L'agent convertit le plan affiné en beads avec dépendances.
09 · VERIFY-BEADS	3 passes séquentielles comparent le plan aux beads et comblent les gaps.

Ce que Gas Town change

	PROBLÈME	RÉPONSE GAS TOWN
	Les workflows single-agent cassent quand le développement devient un enjeu de delivery.	**Transformer le delivery en modèle opérationnel distribué et supervisé.**
01	**Saturation du contexte** Les contraintes importantes disparaissent la session grandit.	Structurer le travail en beads, artefacts et passages de relais explicites.
02	**Exécution séquentielle** Le débit est limité par une seule boucle de worker.	Utiliser les Polecats pour exécuter le travail indépendant en parallèle.
03	**Pas de mécanisme de coordination** Les dépendances et les responsabilités restent implicites.	Mayor, Crew et Convoys rendent responsabilités et progression visibles.
04	**Pression sur le merge** Les changements parallèles deviennent difficiles à intégrer proprement.	La Refinery gère la convergence au lieu de laisser l'intégration au hasard.
05	**Pas de supervision opérationnelle** Le travail bloqué devient difficile à observer et à récupérer.	Witness, Deacon et les boucles de récupération gardent l'exécution observable.

Merci

Questions & discussion

EXEMPLES DE QUESTIONS

Combien coûte le delivery d'une feature avec Gas Town?

Quel niveau de contrôle avons-nous sur les modèles?

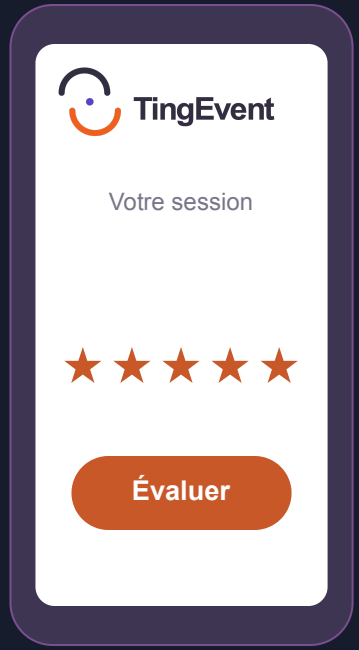
Quand faut-il utiliser le Crew plutôt que seulement le Mayor?

Votre avis compte

Évaluez cette session

Rendez-vous sur l'application mobile **TingEvent** pour partager votre retour en quelques secondes.

1. Ouvrez l'application TingEvent sur votre mobile
2. Rendez-vous sur cette session dans le programme
3. Attribuez votre note et laissez un commentaire



AgilenSeine